**Atividade Prática - Aula 03**

# Persistência de Dados e Primeiro Cadastro Real (Trilhas API + Frontend)

**Baseado em:** roteiro da **Aula 03 — trilha API (Shortz-App API)** e roteiro da **Aula 03 — trilha Frontend (Shortz-App)**

**Aplicação:** este enunciado vale para **qualquer** projeto sorteado na *Lista de Projetos para Programação Web* — use a mesma ficha de preparação que você preencheu e atualizou nas Atividades das Aulas 01 e 02. Não é preciso recriá-la, só acrescentar as linhas novas pedidas na Seção 2.

**Pré-requisito:** Atividades das Aulas 01 e 02 concluídas nas duas trilhas — API com `middlewares/apiResponse.js`, `routes/index.js` (`GET /api`), CORS configurado, e o módulo `modules/search/` já seguindo o padrão **Route → Controller → Service**; front com Vue Router, layout base, `services/api.js` (com interceptor de resposta) e os services `authService.js` / `searchService.js` / `systemService.js` já criados.

> ⚠️ **Ordem de execução:** faça a **Parte A (API) antes da Parte B (Frontend)**, como nas atividades anteriores — a tela de Registro só pode ser testada de ponta a ponta com o cadastro de verdade já funcionando no seu back-end.

## 1. Objetivo

Ao final desta atividade, você deve ter, no seu próprio projeto:

**Do lado do back-end (API):**

- Conexão real com **MySQL via Sequelize** (`config/database.js`), com `sync({ alter: true })` criando as tabelas automaticamente;
- O primeiro **Model com persistência real** (`modules/user/userModel.js`), representando a tabela de usuários do seu sistema;
- **Validação declarativa de entrada** (`express-validator`) no cadastro;
- **Hashing de senha** com `bcryptjs` — a senha nunca é salva em texto puro;

- **Tratamento de erro assíncrono centralizado** ( `middlewares/asyncHandler.js + middlewares/errorHandler.js` ) — necessário a partir de hoje, porque `POST /register` é a primeira rota da sua API com uma operação verdadeiramente assíncrona (consulta ao banco);
- `POST /api/register` funcionando de ponta a ponta, criando um usuário de verdade no seu banco;
- `GET /api/profile/:username` , pública, para confirmar o cadastro por fora do navegador.

**Do lado do front-end:**

- A tela de **Registro** funcional: formulário controlado, validação client-side espelhando as regras do seu `userValidator.js` , envio via `authService.register()` (criada na Aula 02, usada pela primeira vez hoje), feedback de sucesso/erro, e redirecionamento para o Login;
- Um link "**Criar Conta**" navegável na sua Navbar (não custa conferir agora — é um descuido comum passar batido).

Você também deve ser capaz de **explicar com suas palavras**:

- O que é um ORM, e o que `sync({ alter: true })` faz de fato no seu banco;
- Por que a senha nunca é salva em texto puro, e como `bcrypt.hash` / `bcrypt.compare` resolvem isso sem nunca "desfazer" o hash;
- Por que uma função `async` que lança erro precisa de `asyncHandler` , enquanto uma função síncrona não precisa;
- Por que existe validação **nos dois lados** (client e server), e qual delas realmente decide se o dado é aceito.

## 2. Antes de começar

Recupere a sua ficha de preparação das atividades anteriores. Hoje ela ganha duas linhas novas:

| Dado | Seu valor |
|---|---|
| Nome do banco de dados MySQL do seu projeto | `<seu_projeto_em_minusculo>_db` (ex.: `podwave_db` ) |
| Nome do campo de contagem de itens publicados (plural, camelCase) | *(o que substitui `videosCount` : `episodesCount` , `recipesCount` , `workoutsCount` , `postsCount` ...)* |

> 💡 O campo de contagem representa "quantos itens da sua entidade principal este usuário publicou" — em todo projeto da lista existe alguma forma de conteúdo publicado por usuário (aula, episódio, receita, treino registrado, foto, review etc.), então esse campo sempre faz sentido, mesmo que o nome mude.

# PARTE A — Backend: persistência de dados e primeiro cadastro real

## 3. Contextualização rápida

- Até a Aula 02, sua API nunca precisou **guardar** nada. Hoje isso muda: `POST /register` precisa criar uma linha nova numa tabela, e essa linha precisa sobreviver a um reinício do servidor.
- Um **ORM** (*Object-Relational Mapper*) — no nosso caso, o **Sequelize** — representa uma tabela como uma classe JavaScript (o *Model*) e cada linha como uma instância dessa classe, escondendo o SQL por trás de métodos como `User.create(...)`. `sequelize.sync({ alter: true })` cria/ajusta a tabela automaticamente a partir da definição do Model, sem que ninguém escreva `CREATE TABLE` manualmente.
- Senha nunca é salva em texto puro: usamos **hashing** com `bcryptjs` — gerar um `salt` (valor aleatório) e então hashear a senha usando esse salt. No login (próxima aula), comparamos a senha digitada com o hash salvo usando uma função de comparação específica, nunca "desfazendo" o hash.
- Validação declarativa com `express-validator`: uma cadeia de validadores por campo, mais compacta do que checar campo a campo manualmente com `if`.
- **Por que hoje, pela primeira vez, você precisa de tratamento de erro assíncrono:** um erro lançado **dentro de uma função `async`** não é capturado automaticamente pelo Express 4 — a requisição ficaria pendurada para sempre, sem resposta. Por isso, toda rota que chamar uma função `async` a partir de hoje precisa passar por um middleware auxiliar (`asyncHandler`), e todo erro capturado por ele precisa de um destino final (`errorHandler`).

## 4. Etapa 1 — Instalando as dependências de hoje

```
npm install sequelize mysql2 bcryptjs express-validator
```

- `sequelize` → o ORM em si.
- `mysql2` → o *driver* que conversa com o MySQL de verdade; o Sequelize usa ele por baixo dos panos.
- `bcryptjs` → hashing de senha (versão "JS pura", sem dependência de compilação nativa).
- `express-validator` → validação declarativa de entrada.

**Checklist desta etapa:**

- [ ] `sequelize`, `mysql2`, `bcryptjs`, `express-validator` instalados

## 5. Etapa 2 — Preparando o Banco de Dados MySQL

Crie um schema novo, exclusivo do seu projeto:

```
CREATE DATABASE <seu_banco_de_dados>;
```

Edite o `.env` da sua API, acrescentando as credenciais do banco:

```
PORT=<sua_porta>

CORS_ORIGIN=http://localhost:5173

DB_HOST=localhost
DB_PORT=3306
DB_NAME=<seu_banco_de_dados>
DB_USER=root
DB_PASSWORD=coloque_aqui_a_senha_do_seu_mysql
```

Crie `config/database.js` (código idêntico para qualquer projeto):

```
const { Sequelize } = require('sequelize');
require('dotenv').config();

const sequelize = new Sequelize(
    process.env.DB_NAME,
    process.env.DB_USER,
    process.env.DB_PASSWORD,
    {
        host: process.env.DB_HOST,
        port: process.env.DB_PORT,
        dialect: 'mysql',
        logging: false,
        define: {
            timestamps: true,
            underscored: true
        }
    }
);

module.exports = sequelize;
```

**Checklist desta etapa:**

- [ ] Banco de dados criado no MySQL, com o nome anotado na sua ficha
- [ ] `.env` atualizado com `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`
- [ ] `config/database.js` criado

## 6. Etapa 3 — `config/constants.js`

Crie `config/constants.js`:

```
module.exports = {
    VALIDATION: {
        USERNAME_MIN: 3,
        USERNAME_MAX: 20,
        PASSWORD_MIN: 6
    }
};
```

Esses valores centralizam as regras de validação de hoje — o front vai replicar exatamente estes mesmos números na Parte B, então não altere sem atualizar os dois lados.

**Checklist desta etapa:**

- [ ] `config/constants.js` criado

## 7. Etapa 4 — O Módulo `user`

```
mkdir modules\user
```

### 7.1. `modules/user/userModel.js`

```
const { DataTypes } = require('sequelize');
const sequelize = require('../../config/database');

const User = sequelize.define('User',
    {
        id:             { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true },
        username:       { type: DataTypes.STRING, allowNull: false, unique: 'idx_unique_username' },
        email:          { type: DataTypes.STRING, allowNull: false, unique: 'idx_unique_email', validate: { isEmail: true } },
        password:       { type: DataTypes.STRING, allowNull: false },
```

```
        fullName:       { type: DataTypes.STRING, allowNull: true },
        bio:            { type: DataTypes.STRING(255), allowNull: true },
        profilePicture: { type: DataTypes.STRING, allowNull: true, defaultValue: 'default-profile.png' },
        followersCount: { type: DataTypes.INTEGER, defaultValue: 0 },
        followingCount: { type: DataTypes.INTEGER, defaultValue: 0 },
        <seuCampoDeContagem>: { type: DataTypes.INTEGER, defaultValue: 0 },
        isBlocked:      { type: DataTypes.BOOLEAN, defaultValue: false },
        isAdmin:        { type: DataTypes.BOOLEAN, defaultValue: false }
    },
    {
        timestamps: true,
        tableName: 'users',
        indexes: [
            { unique: true, fields: ['username'], name: 'idx_unique_username' },
            { unique: true, fields: ['email'], name: 'idx_unique_email' }
        ]
    }
);

module.exports = User;
```

> ⚠️ Troque `<seuCampoDeContagem>` pelo valor anotado na sua ficha (Seção 2).

**Por que o Model já nasce com todas essas colunas, se hoje só usamos quatro delas:** a estrutura nasce inteira, o conteúdo é ativado aula a aula. `bio` / `profilePicture` são ativados quando vocês construírem a tela de edição de perfil; `followersCount` / `followingCount` quando o módulo de seguir usuários existir; `<seuCampoDeContagem>` quando o upload/criação de conteúdo existir; `isAdmin` entra em uso já na próxima aula (vai para dentro do token JWT); `isBlocked` fica adormecido até a aula de painel administrativo.

### 7.2. `modules/user/userValidator.js`

```
const { body, validationResult } = require('express-validator');
const { VALIDATION } = require('../../config/constants');
```

```
const validate = (req, res, next) => {
    const errors = validationResult(req);
    if (errors.isEmpty()) {
        return next();
    }

    const firstError = errors.array()[0].msg;

    const error = new Error(firstError);
    error.status = 400;
    error.errors = errors.array();
    throw error;
};

exports.registerValidator = [
    body('username')
        .isLength({ min: VALIDATION.USERNAME_MIN, max: VALIDATION.USERNAME_MAX })
        .withMessage(`O nome de usuário deve ter entre ${VALIDATION.USERNAME_MIN} e ${VALIDATION.USERNAME_MAX} caracteres.`)
        .trim(),
    body('email')
        .isEmail()
        .withMessage('Por favor, insira um e-mail válido.')
        .normalizeEmail(),
    body('password')
        .isLength({ min: VALIDATION.PASSWORD_MIN })
        .withMessage(`A senha deve ter pelo menos ${VALIDATION.PASSWORD_MIN} caracteres.`),
    body('confirmPassword')
        .custom((value, { req }) => {
            if (value !== req.body.password) {
                throw new Error('As senhas não coincidem.');
            }
            return true;
        }),
    body('fullName')
        .notEmpty()
```

```
        .withMessage('O nome completo é obrigatório.')
        .trim(),
    validate
];
```

Este arquivo é **idêntico** para qualquer projeto — as regras de cadastro (usuário, e-mail, senha, confirmação, nome completo) são universais.

### 7.3. `modules/user/userService.js`

```
const User = require('./userModel');
const bcrypt = require('bcryptjs');

async function registerUser(username, email, password, fullName) {
    const emailExists = await User.findOne({ where: { email } });
    const usernameExists = await User.findOne({ where: { username } });
    if (emailExists || usernameExists) {
        throw new Error('Este e-mail ou usuário já está cadastrado.');
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);

    const newUser = await User.create({
        username,
        email,
        password: hashedPassword,
        fullName
    });

    return {
        id: newUser.id,
        username: newUser.username,
        email: newUser.email
```

```
    };
}

async function getPublicProfile(username) {
    const user = await User.findOne({
        where: { username },
        attributes: ['id', 'username', 'fullName', 'bio', 'profilePicture', 'followersCount', 'followingCount', '<seuCampoDeContagem>'
    });

    if (!user) {
        const error = new Error('Usuário não encontrado.');
        error.status = 404;
        throw error;
    }

    return user;
}

module.exports = { registerUser, getPublicProfile };
```

> ⚠️ Troque `<seuCampoDeContagem>` pelo nome da sua ficha, igual ao Model.

> ⚠️ **Atenção ao status do erro de duplicidade:** repare que `if (emailExists || usernameExists) { throw new Error(...) }` **não** define `error.status`. Isso significa que esse erro cai no `500` padrão do `errorHandler` (Etapa 5), não num `400` / `409` como seria semanticamente mais correto. **Não corrija isso** — o roteiro de frontend desta atividade já foi escrito considerando esse comportamento.

**Checklist desta etapa (Model, Validator, Service):**

- [ ] `userModel.js` criado, com o campo de contagem renomeado corretamente
- [ ] `userValidator.js` criado
- [ ] `userService.js` criado, com `getPublicProfile` também usando o campo de contagem renomeado
- [ ] Nunca a senha (nem o hash) é devolvida em nenhuma resposta

### 7.4. `middlewares/asyncHandler.js`

```
module.exports = (fn) => (req, res, next) => {
    Promise.resolve(fn(req, res, next)).catch(next);
};
```

### 7.5. `middlewares/errorHandler.js`

```
const { error } = require('./apiResponse');

module.exports = (err, req, res, next) => {
    console.error(err);

    const statusCode = err.status || 500;
    const errors = err.errors || [];

    return error(res, err.message || 'Ocorreu um erro inesperado.', statusCode, errors);
};
```

Estes dois arquivos são **idênticos** para qualquer projeto — não conhecem nada do seu domínio.

**Checklist desta etapa:**

- [ ] `middlewares/asyncHandler.js` criado
- [ ] `middlewares/errorHandler.js` criado

### 7.6. `modules/user/userController.js`

```
const userService = require('./userService');
const { success } = require('../../middlewares/apiResponse');
```

```
exports.register = async (req, res) => {
    const { username, email, password, fullName } = req.body;

    const newUser = await userService.registerUser(username, email, password, fullName);

    return success(res, newUser, 'Conta criada com sucesso! Faça login para continuar.', 201);
};

exports.getPublicProfile = async (req, res) => {
    const user = await userService.getPublicProfile(req.params.username);

    return success(res, user);
};
```

### 7.7. `modules/user/userRoutes.js`

```
const express = require('express');
const router = express.Router();
const userController = require('./userController');
const { registerValidator } = require('./userValidator');
const asyncHandler = require('../../middlewares/asyncHandler');

router.post('/register', registerValidator, asyncHandler(userController.register));

router.get('/profile/:username', asyncHandler(userController.getPublicProfile));

module.exports = router;
```

### Checklist desta etapa:

- [ ] `userController.js` e `userRoutes.js` criados, ambos idênticos ao modelo (nenhuma adaptação necessária aqui)

## 8. Etapa 5 — Registrando o Módulo e o `errorHandler` em `app.js`

Edite `app.js`:

```
var express = require('express');
var logger = require('morgan');
var cors = require('cors');
require('dotenv').config();

var indexRouter = require('./routes/index');
var searchRoutes = require('./modules/search/searchRoutes');
var userRoutes = require('./modules/user/userRoutes');          // ADICIONE ESTA LINHA
var errorHandler = require('./middlewares/errorHandler');       // ADICIONE ESTA LINHA

var app = express();

app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

app.use(cors({
    origin: process.env.CORS_ORIGIN || 'http://localhost:5173',
    credentials: true
}));

app.use('/api', indexRouter);
app.use('/api', searchRoutes);
app.use('/api', userRoutes);                                    // ADICIONE ESTA LINHA

app.use((req, res) => {
    res.status(404).json({
        success: false,
        message: 'Rota não encontrada.',
        errors: []
```

```
    });
});

app.use(errorHandler);                                          // ADICIONE ESTA LINHA — sempre o ÚLTIMO app.use()

const sequelize = require('./config/database');
sequelize.sync({ alter: true })
    .then(() => console.log('Banco de dados sincronizado!'))
    .catch(err => console.error('Erro ao sincronizar banco:', err));

module.exports = app;
```

> ⚠️ **Ordem importa:** `app.use(errorHandler)` precisa vir **depois** do middleware de 404, e ser o último `app.use()` do arquivo — um middleware de erro só é alcançado via `next(err)`, nunca pela ordem normal de execução.

Reinicie a API e confirme, no terminal, a mensagem `Banco de dados sincronizado!`.

**Checklist desta etapa:**

- [ ] `userRoutes` e `errorHandler` importados e registrados em `app.js`, na ordem certa
- [ ] Terminal exibe `Banco de dados sincronizado!` ao subir a API
- [ ] No MySQL, a tabela `users` do seu banco existe, com todas as colunas do Model

## 9. Etapa 6 — Testando

```
# Cadastro com sucesso
curl -X POST http://localhost:3000/api/register \
  -H "Content-Type: application/json" \
  -d '{"username":"teste123","email":"teste@email.com","password":"123456","confirmPassword":"123456","fullName":"Usuário de Teste"}'
```

Deve responder `201` :

```
{"success":true,"message":"Conta criada com sucesso! Faça login para continuar.","data":{"id":1,"username":"teste123","email":"teste@e
```

```
# Erro de validação (senha curta) — deve responder 400
curl -X POST http://localhost:3000/api/register \
  -H "Content-Type: application/json" \
  -d '{"username":"pedro","email":"pedro@email.com","password":"123","confirmPassword":"123","fullName":"Pedro Souza"}'

# Erro de duplicidade — rode o MESMO comando do cadastro de sucesso outra vez — deve responder 500
curl -X POST http://localhost:3000/api/register \
  -H "Content-Type: application/json" \
  -d '{"username":"teste123","email":"teste@email.com","password":"123456","confirmPassword":"123456","fullName":"Usuário de Teste"}'

# Confirmando o cadastro por fora
curl http://localhost:3000/api/profile/teste123
```

**Checklist desta etapa:**

- [ ] Cadastro com sucesso responde `201` com `{ id, username, email }`
- [ ] Senha curta responde `400` com a mensagem correta
- [ ] Cadastro duplicado responde `500` (não é bug, é esperado)
- [ ] `GET /profile/:username` confirma os dados persistidos

## PARTE B — Frontend: a Tela de Registro

## 10. Contextualização rápida

- A API já valida os dados recebidos — mas isso não dispensa a validação no front. A validação do **servidor** é a que realmente decide se o dado é aceito (nunca pode ser removida, porque qualquer um pode chamar a API diretamente, pulando sua tela). A validação do **cliente** existe por experiência do usuário: o erro aparece instantaneamente, sem gastar uma chamada de rede.
- **Regra de ouro:** a validação do front é uma cópia otimista das regras do backend, nunca a única linha de defesa.

## 11. Etapa 1 — Conhecendo o Contrato da Sua API

| Campo | Regra |
|---|---|
| `username` | entre 3 e 20 caracteres |
| `email` | precisa ser um e-mail válido |
| `password` | mínimo 6 caracteres |
| `confirmPassword` | precisa ser idêntico a `password` |
| `fullName` | obrigatório |

**Resposta de sucesso** ( `201` ): `{ success: true, message: "...", data: { id, username, email } }` .

**Resposta de erro de duplicidade:** `message: "Este e-mail ou usuário já está cadastrado."` , com status `500` — não trate esse caso como especial: o interceptor da Aula 02 já entrega `error.message` independente do status.

## 12. Etapa 2 — Conferindo a Navbar

Antes de construir a tela, confira `src/components/layout/TheNavbar.vue` : existe algum jeito de um visitante **não logado** chegar até `/register` clicando em algo (não só digitando a URL)? Se não existir, acrescente:

```
<router-link to="/register">Criar Conta</router-link>
```

> 💡 Este é um lembrete que vale para **toda** tela nova daqui para frente: sempre confira se ela é alcançável por navegação de verdade, não só por URL digitada manualmente.

**Checklist desta etapa:**

- [ ] Existe um link clicável para `/register` em algum lugar acessível a um visitante deslogado

## 13. Etapa 3 — Construindo o Formulário Controlado

Edite `src/views/auth/RegisterView.vue` :

```
<script setup>
import { reactive, ref } from 'vue'
import { useRouter } from 'vue-router'
import { register } from '../../services/authService'

const router = useRouter()

const form = reactive({
  username: '',
  email: '',
  password: '',
  confirmPassword: '',
  fullName: '',
})

const errors = reactive({
  username: '',
  email: '',
  password: '',
```

```
  confirmPassword: '',
  fullName: '',
})

const isSubmitting = ref(false)
const apiErrorMessage = ref('')
</script>

<template>
  <div class="auth-page">
    <h1>Criar Conta</h1>

    <form @submit.prevent="handleSubmit">
      <div class="field">
        <label for="fullName">Nome completo</label>
        <input id="fullName" type="text" v-model="form.fullName" />
        <span v-if="errors.fullName" class="error-message">{{ errors.fullName }}</span>
      </div>

      <div class="field">
        <label for="username">Usuário</label>
        <input id="username" type="text" v-model="form.username" />
        <span v-if="errors.username" class="error-message">{{ errors.username }}</span>
      </div>

      <div class="field">
        <label for="email">E-mail</label>
        <input id="email" type="email" v-model="form.email" />
        <span v-if="errors.email" class="error-message">{{ errors.email }}</span>
      </div>

      <div class="field">
        <label for="password">Senha</label>
        <input id="password" type="password" v-model="form.password" />
        <span v-if="errors.password" class="error-message">{{ errors.password }}</span>
```

```
      </div>

      <div class="field">
        <label for="confirmPassword">Confirmar senha</label>
        <input id="confirmPassword" type="password" v-model="form.confirmPassword" />
        <span v-if="errors.confirmPassword" class="error-message">{{ errors.confirmPassword }}</span>
      </div>

      <p v-if="apiErrorMessage" class="api-error">{{ apiErrorMessage }}</p>

      <button type="submit" :disabled="isSubmitting">
        {{ isSubmitting ? 'Criando conta...' : 'Criar Minha Conta' }}
      </button>
    </form>
  </div>
</template>
```

**Checklist desta etapa:**

- [ ] Formulário controlado com `v-model` amarrando cada campo a `form`
- [ ] `errors` e `apiErrorMessage` já preparados no `<script setup>`, mesmo antes de escrever `validate()` / `handleSubmit()`

## 14. Etapa 4 — Validação Client-Side

Complete o `<script setup>` de `RegisterView.vue`, acrescentando a função de validação — os números abaixo **precisam bater** com o que você definiu em `config/constants.js` na Parte A:

```
function validate() {
  Object.keys(errors).forEach((key) => (errors[key] = ''))

  if (!form.fullName.trim()) {
    errors.fullName = 'O nome completo é obrigatório.'
```

```
  }

  if (form.username.trim().length < 3 || form.username.trim().length > 20) {
    errors.username = 'O nome de usuário deve ter entre 3 e 20 caracteres.'
  }

  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
  if (!emailPattern.test(form.email)) {
    errors.email = 'Informe um e-mail válido.'
  }

  if (form.password.length < 6) {
    errors.password = 'A senha deve ter pelo menos 6 caracteres.'
  }

  if (form.confirmPassword !== form.password) {
    errors.confirmPassword = 'As senhas não coincidem.'
  }

  return Object.values(errors).every((message) => message === '')
}
```

Adicione, no `<template>`, um `<span v-if="errors.CAMPO" class="error-message">{{ errors.CAMPO }}</span>` logo abaixo de cada campo (se ainda não fez isso na Etapa 3).

**Checklist desta etapa:**

- [ ] `validate()` replica exatamente os mesmos limites configurados no `userValidator.js` do seu back-end
- [ ] Mensagens de erro aparecem/desaparecem corretamente conforme o usuário corrige os campos

## 15. Etapa 5 — Integrando com a API

```
async function handleSubmit() {
  apiErrorMessage.value = ''

  if (!validate()) {
    return
  }

  isSubmitting.value = true

  try {
    const response = await register({
      username: form.username.trim(),
      email: form.email.trim(),
      password: form.password,
      confirmPassword: form.confirmPassword,
      fullName: form.fullName.trim(),
    })

    console.log('Conta criada:', response.data)
    router.push({ name: 'login' })
  } catch (error) {
    apiErrorMessage.value = error.message
  } finally {
    isSubmitting.value = false
  }
}
```

**Checklist desta etapa:**

- [ ] Envio vazio não dispara nenhuma chamada de rede (confira na aba Network)
- [ ] Cadastro válido navega para `/login`
- [ ] Cadastro duplicado exibe `apiErrorMessage` com a mensagem vinda de verdade da API

## 16. Etapa 6 — Teste Prático e Confirmação Fora do Navegador

1. Envie o formulário vazio → confirme os erros de cada campo, sem chamada de rede.
2. Preencha com senha curta → confirme a mensagem instantânea.
3. Preencha corretamente e envie → confirme a navegação para `/login`.
4. Tente cadastrar o mesmo e-mail/usuário de novo → confirme `apiErrorMessage` com a mensagem de duplicidade (Network mostrando `500`).
5. Confirme por fora do navegador:

```
curl http://localhost:3000/api/profile/<username-que-voce-cadastrou>
```

**Checklist desta etapa:**

- [ ] Os cinco comportamentos acima foram testados e conferem
- [ ] `curl` confirma que o cadastro feito pela tela está persistido no banco

# 17. Entrega

Envie em resposta a esta tarefa:

1. Na subpasta `01 - Docs` do seu projeto frontend, dentro de uma nova subpasta `atividade03`:
   - A ficha em markdown, atualizada com as duas linhas novas da Seção 2;
   - Um print do MySQL (Workbench/DBeaver/linha de comando) mostrando a tabela `users` criada, em `.jpg`, com o nome `tabela-users.jpg`;
   - Um print de cada um dos quatro `curl` da Etapa 6 da Parte A, em um único arquivo ou em quatro arquivos `.jpg` nomeados `curl-cadastro-sucesso.jpg`, `curl-erro-validacao.jpg`, `curl-erro-duplicidade.jpg`, `curl-profile.jpg`;

    - Um print do formulário de Registro exibindo os erros de validação client-side, em `.jpg`, com o nome `registro-erros.jpg`;
    - Um print da aba Network do DevTools mostrando a chamada de cadastro duplicado com status `500`, em `.jpg`, com o nome `registro-duplicidade-network.jpg`;
    - Atualize `checklists.md` com todos os checklists desta atividade marcados.
2. Um arquivo `.zip` do seu projeto de back-end (sem `node_modules`), nomeado `<NomeSobrenome>-<NomeProjeto>-backend.zip`, agora incluindo `config/`, `modules/user/` e os middlewares novos;
3. Um arquivo `.zip` do seu projeto de front-end (sem `node_modules`), nomeado `<NomeSobrenome>-<NomeProjeto>-frontend.zip`, agora incluindo `views/auth/RegisterView.vue` funcional.